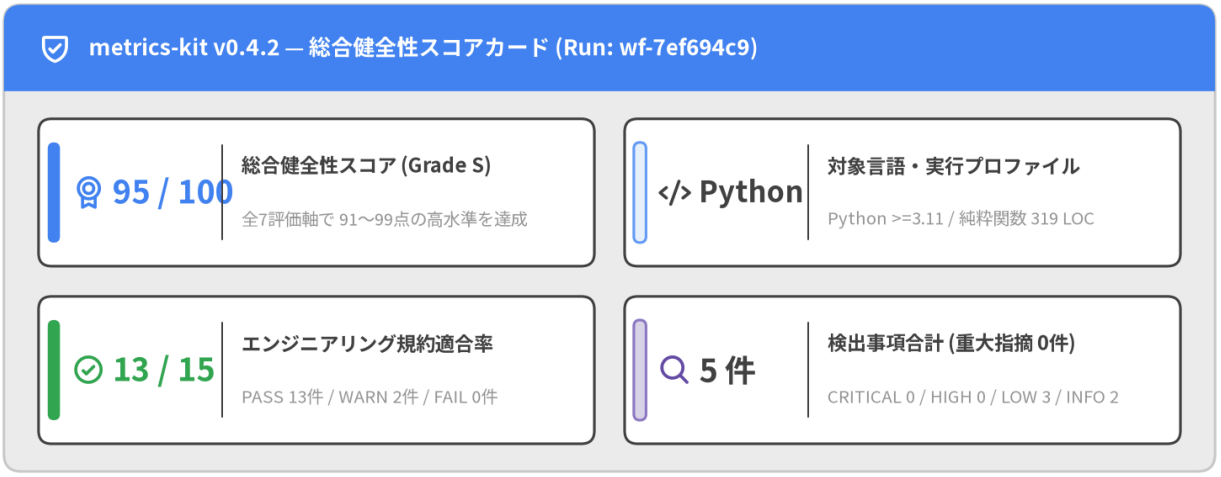
PNI ソフトウェア健全性診断レポート

対象プロジェクト: `metrics-kit` v0.4.2（軽量テレメトリ統計・SLO閾値評価 Python ライブラリ）
診断ワークフロー: `py_lib_clean`（隔離コンテナ動的検証・Trivy 脆弱性スキャン統合フル監査）



PNI 総合健全性スコアカード — metrics-kit v0.4.2 (Grade S: 95/100)

- **診断対象**: `examples/python/lib_clean`（`metrics-kit` `v0.4.2` / Apache-2.0 / `pydantic>=2.7.0`）
- **実行ワークフロー / Run ID**: `py_lib_clean` / `wf-7ef694c9`（評価日時: `2026-10-24T07:36:17Z`）
- **総合判定**: **95 / 100 (Grade S)** — ルール適合 `13/15`・オフライン `pytest` `6/6` 合格（行カバレッジ `96.2%`）・直接依存 CVE `0` 件

Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する変更対応

4. 検査指標・ルール適合マトリクスと指摘一覧

5. 詳細分析：コード品質・テスト・動的検証

6. 詳細分析：セキュリティ・脆弱性・API境界

1. エグゼクティブサマリー

`metrics-kit` (v0.4.2) は総合スコア **95/100 (Grade S)** と極めて高い健全性を示す Python 統計・SLO閾値評価ライブラリである。319 LOC・6モジュールの単方向4層アーキテクチャ全体で型安全性・Import衛生・Docstringカバレッジがいずれも **100.0%** に達し、隔離コンテナでのオフライン単体テスト全件合格と実測行カバレッジ **96.2%** を実証した。

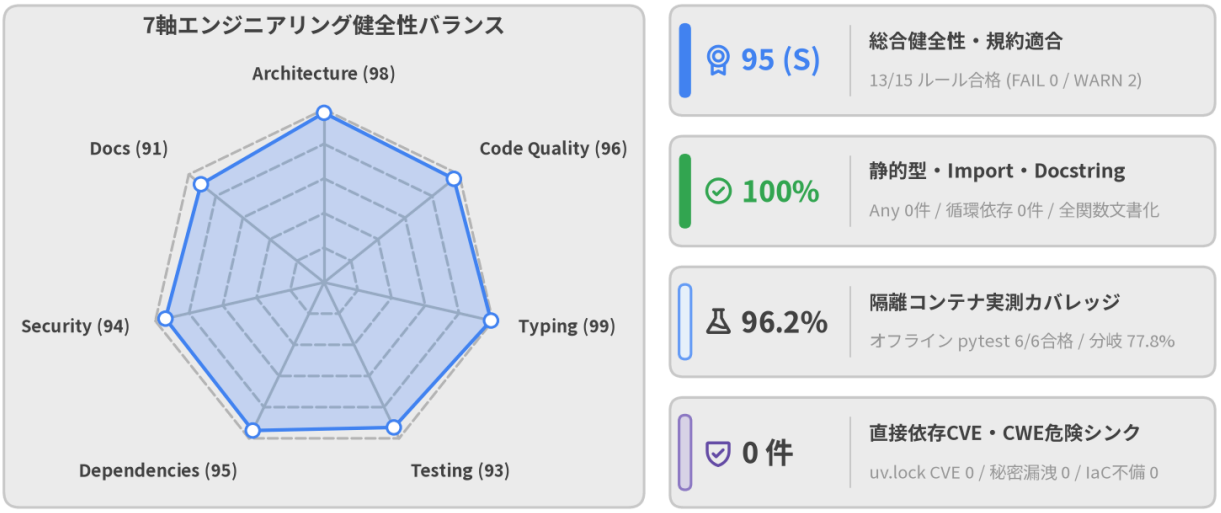


図1-1: 7軸エンジニアリング健全性レーダーと主要ヘルスKPI

1.1 エグゼクティブ総括と7軸スコア内訳

`metrics-kit` は時系列テレメトリデータの記述統計集計、EWMA平滑化、宣言的SLO閾値ルール評価、および Prometheus / Markdown 出力整形を副作用ゼロで提供する模範的な軽量ライブラリである。6つの専門監査ステップすべてで Grade S（`python_test_audit`: 91、`security_vulnerability_audit`: 93、`repo_structure_audit`: 94、`library_hygiene_audit`: 95、`container_runtime_audit`: 95、`python_ast_audit`: 96）を獲得した。

評価軸	スコア	要約根拠
Typing（型安全性）	**99 / 100**	PEP 561 `py.typed` 同梱、型注釈カバレッジ `100.0%`、`Any` / `type: ignore` `0` 件
Architecture（設計）	**98 / 100**	4層単方向DAG構造（循環依存 `0` 件・レイヤ逆転 `0` 件）、`__all__` による厳格な公開境界
Code Quality（品質）	**96 / 100**	高複雑度関数 `0` 件、グローバル可変状態 `0` 件、Docstring `100.0%`（NaN/Inf境界のみ改善余地）
Dependencies（依存）	**95 / 100**	実行時依存は `pydantic>=2.7.0` の1件のみ、Dependabot週次更新・SHA固定CIを完備
Security（安全性）	**94 / 100**	`uv.lock` CVE `0` 件、CWE危険シンク `0` 件、秘密漏洩 `0` 件（ベースイメージ `pip` に中程度CVE）
Testing（テスト）	**93 / 100**	オフライン `pytest` `6/6` 全件合格、行カバレッジ `96.2%` / 分岐カバレッジ `77.8%`
Documentation（文書）	**91 / 100**	Docstring `100.0%`、4象限構成の `README.md`（公開関数1件の追記と開発手順整備を推奨）

1.2 主要な強みと優先対応リスクの対比



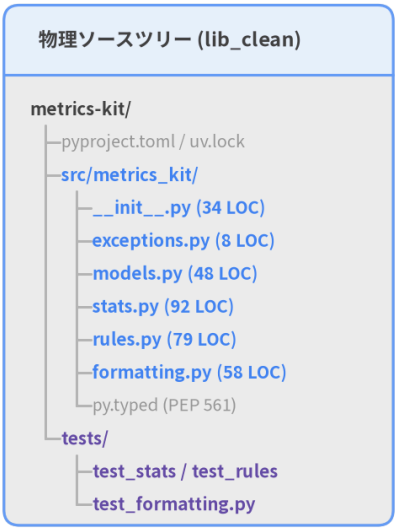
図1-2: 確認された主要な強み (Strengths) と優先対応リスク (Top Risks)

さらなる完成度向上に向け、**P1**（優先度：高）として `NaN` / `Infinity` 非有限浮動小数点数の境界ガード追加および `uv.lock` 推移的ハッシュ完全化・コンテナ検証用ベースイメージ更

新、**P2**（優先度：中）として CI への `ty check` / `--locked` 追加、`README.md` 整備、および境界条件 `parametrize` テストの拡充を推奨する（詳細は第3章参照）。

2. アーキテクチャとソースコード構成

`metrics-kit` は単方向4層レイヤード・アーキテクチャ（Facade → Policy & Formatting → Statistical Core → Domain Models & Exceptions）を採用した純粋インメモリ Python ライブラリである。全6モジュール・319 LOC の内部インポートグラフは完全な有向非巡回グラフ（DAG）を形成し、循環依存やレイヤ逆転は 0 件である。



モジュール	LOC	定義クラス・型	公開関数・主要責務
__init__.py	34	再公開 19シンボル	__all__ パッケージ公開境界
exceptions.py	8	MetricsKitError 他2	ドメイン例外階層 (空・不正引数)
formatting.py	58	-	Prometheus / Markdown 整形3関数
models.py	48	MetricPoint 他3型	Pydantic v2 frozen 不変モデル
rules.py	79	-	SLO閾値評価・Zスコア異常検知3関数
stats.py	92	-	平均・標準偏差・分位・EWMA 5関数

図2-1: 物理ディレクトリツリーと AST モジュールインベントリ (全6ファイル・計319 LOC)

2.1 システムプロファイルと単方向4層レイヤ構造

- **ランタイム種別・技術スタック**: 純粋インメモリ Python ライブラリ（`Python >=3.11`、`Pydantic >=2.7.0`、`uv_build`、`pytest` / `pytest-cov`、`Ruff`、`Ty`）。外部ネットワーク通信やファイルシステム I/O を一切伴わず、Prometheus Text Exposition 形式および Markdown サマリ文字列を純粋関数で生成する。
- **レイヤ責務と境界検証**:
  1. **Public API Facade Layer (`src/metrics_kit/__init__.py`)**: 公開クラス・関数・例外の計19シンボルを `__all__` で明示再エクスポートする。
  2. **Policy & Formatting Layer (`src/metrics_kit/rules.py`, `src/metrics_kit/formatting.py`)**: SLO閾値ルール評価・Zスコア異常値検出、および Prometheus / Markdown 文字列整形を担う。

3. **Statistical Aggregation Layer (`src/metrics_kit/stats.py`)**: 算術平均・母標準偏差・線形補間パーセンタイル・EWMA平滑化・時系列サマリ生成を純粋関数で実行する。
4. **Domain Models & Exceptions Layer (`src/metrics_kit/models.py`, `src/metrics_kit/exceptions.py`)**: `ConfigDict(frozen=True, extra="forbid")` による不変モデルと型付き例外階層を定義し、内部依存を持たない基底層を構成する。

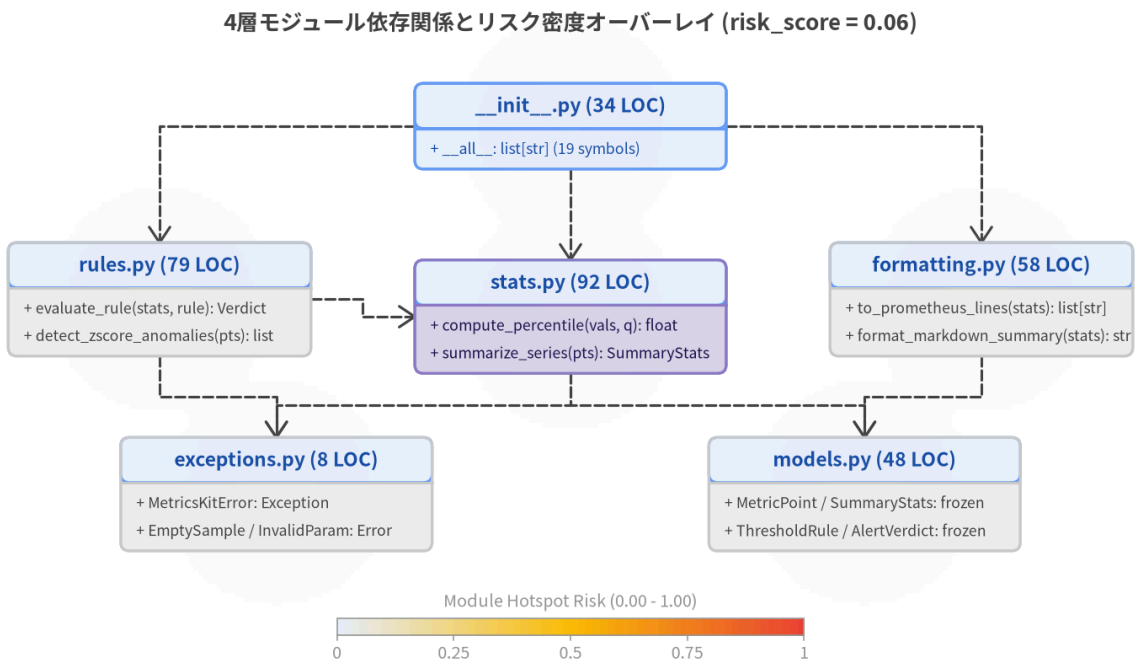


図2-2: 4層モジュール依存グラフ (ClassDiagram) とホットスポットリスク分布 (HeatmapOverlay: 全モジュール risk=0.06)

2.2 主要実行フローと並行・状態管理モデル

公開エントリポイント（`src/metrics_kit/__init__.py`、`stats.py`、`rules.py`、`formatting.py`）はすべてステートレスかつスレッドセーフに動作する。全データモデルが `frozen=True` で不変であり、異常時は `sys.exit` や標準出力汚染を行わず `EmptySampleError` / `InvalidParameterError` を送出する。

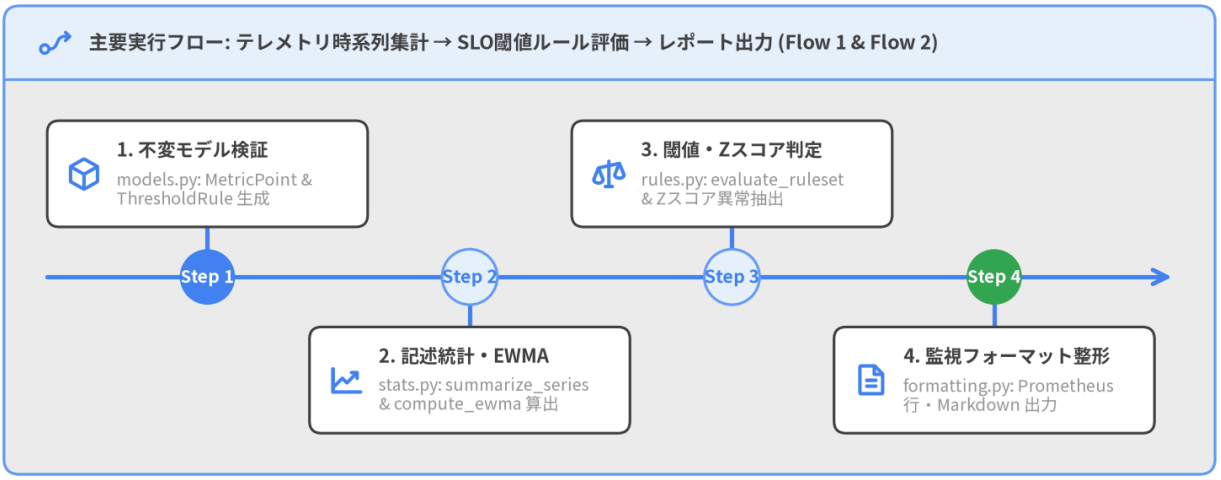


図2-3: 主要ランタイム実行フロー (SLO閾値評価パイプラインおよびEWMA・異常値検出フロー)

3. 推奨する変更対応

本診断では緊急対応を要するブロッカー（`P0`）は **0 件**であった。ライブラリの数値境界安全性とサプライチェーン再現性をさらに高める `P1` **アクション 2 件**、および CI・ドキュメント・境界テストを完璧に仕上げる `P2` **アクション 3 件**の段階的適用を推奨する。

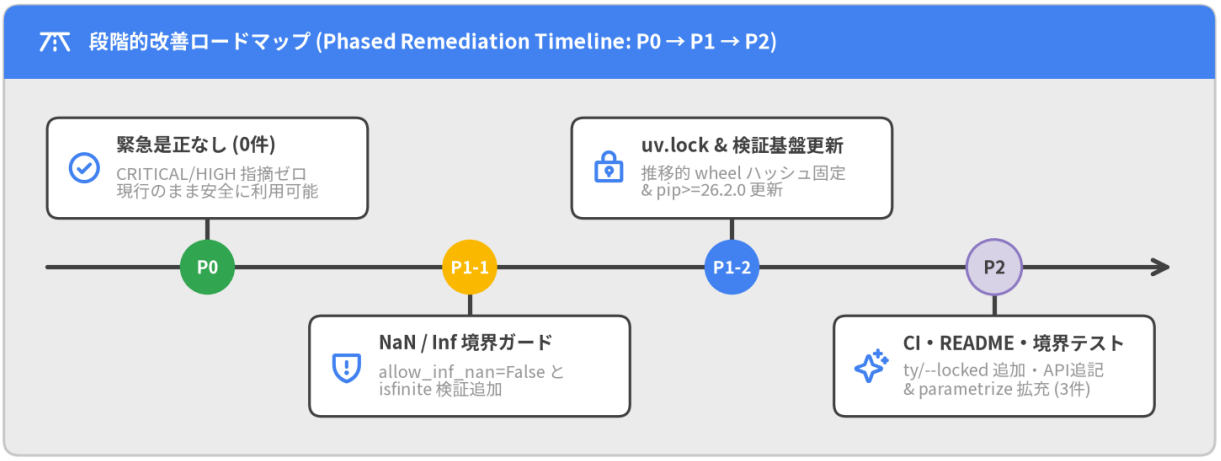


図3-1: フェーズ別改善ロードマップ (P0: 0件 → P1: 高優先2件 → P2: 品質研磨3件)

3.1 アクションプラン・トレーサビリティマトリクス

以下の図3-2および詳細リストは、検出された指摘事項と関連規約・CVE ID に対する全5件の推奨アクションを優先度順（`P1 → P2`）に整理したものである。

優先度	推奨アクション項目	対象ファイル	期待される改善効果	関連ルール / CVE
P1	NaN / Infinity 非有限値の境界バリデーション追加	models.py, stats.py, tests/test_stats.py	非有限値混入時のソート破綻防止と例外契約の一貫性保証	common/security, python/code_quality
P1	uv.lock 推移的ハッシュ完全化と検証ベース環境 pip 更新	uv.lock, pyproject.toml	決定論的ハッシュ検証の確立とコンテナ pip CVE 5件解消	security_vulnerability, CVE-2026-13346 他
P2	CI への ty 型チェックと --locked 検証フラグ追加	.github/workflows/ci.yml, pyproject.toml	PR段階でのロック乖離防止と静的型安全性の自動ゲート化	common/supply_chain, python/dependencies
P2	README への公開API追記と開発・テスト手順の整備	README.md	公開19シンボルとの完全一致と開発者オンボーディング向上	common/documentation
P2	境界条件エッジケースの parametrize テスト拡充	test_stats.py, test_rules.py, test_formatting.py	行・分岐カバレッジの98%〜100%到達と境界回帰耐性の強化	python/testing

図3-2: アクションプラン・トレーサビリティマトリクス (優先度順 P1 → P2)

3.2 各アクションの具体的実装ガイド

1. `[P1]` **NaN / Infinity 非有限浮動小数点数に対する境界バリデーション追加**（分類: `code_quality`）
   - **対象ファイル**: `src/metrics_kit/models.py`, `src/metrics_kit/stats.py`, `tests/test_stats.py`
   - **実施内容**: `models.py` の各モデル `ConfigDict` に `allow_inf_nan=False` を設定するとともに、`stats.py` の `_require_non_empty` で `math.isfinite()` を検証し、非有限値に対して `InvalidParameterError` を送出する単体テストを追加する。
   - **期待効果**: 異常テレメトリ値（`NaN` / `Inf`）混入時にも未処理の `ValueError` や不正確なパーセンタイル算出を防ぎ、公開APIの例外契約を一貫して保証できる。
2. `[P1]` `uv.lock` **の推移的依存・SHA256ハッシュ完全化と検証ベースイメージ pip/OS 更新**（分類: `security`）
   - **対象ファイル**: `uv.lock`, `pyproject.toml`
   - **実施内容**: `uv lock --upgrade` で `pydantic` 2.14+ および推移的依存（`pydantic-core`, `annotated-types`, `typing-extensions`）の wheel SHA-256 ハッシュを `uv.lock` に完全記録し、コンテナ検証環境ではシステム `pip>=26.2.0` 更新または不要な `pip` 除去と Debian セキュリティパッチ適用を行う（関連: `CVE-2026-13346`, `CVE-2026-8643`, `CVE-2026-6357`）。

- **期待効果**: 決定論的ハッシュ検証によるサプライチェーン耐性が向上し、コンテナスキャン上の MEDIUM CVE（`pip` 5件）を解消できる。

3. `[P2]` **CI パイプラインへの `ty` 型チェックと `--locked` 検証の追加**（分類: `dependencies`）
   - **対象ファイル**: `.github/workflows/ci.yml`, `pyproject.toml`
   - **実施内容**: `.github/workflows/ci.yml` で `uv run --locked ruff check src tests`、`uv run --locked ty check`、`uv run --locked pytest --cov=metrics_kit` を実行するよう強化する。
4. `[P2]` `README.md` **への公開API追記と開発・テスト手順セクションの整備**（分類: `documentation`）
   - **対象ファイル**: `README.md`
   - **実施内容**: `## API Reference` に `sanitize_prometheus_name` を追記し、`uv run ruff check` / `uv run ty check` / `uv run pytest --cov` の開発者向け手順を明記する。
5. `[P2]` **境界条件エッジケース（単一要素・ゼロ分散・全比較演算子）の `parametrize` テスト拡充**（分類: `testing`）
   - **対象ファイル**: `tests/test_stats.py`, `tests/test_rules.py`, `tests/test_formatting.py`
   - **実施内容**: `ComparisonOp.LT`、`z_threshold <= 0`、サンプル点数 `< 2`、標準偏差 `0`、単一要素 `compute_percentile`、および空白 `metric_name` の各分岐を `pytest.mark.parametrize` で網羅する。

4. 検査指標・ルール適合マトリクスと指摘一覧

`py_lib_clean` ワークフローで実行された全6監査ステップはいずれも **91〜96点**（Grade S）を達成し、評価対象 15 ルールのうち **13 件が PASS、2 件が WARN、0 件が FAIL** となった。検出された指摘事項も重大度 `LOW` 3件・`INFO` 2件の計5件にとどまり、`CRITICAL` / `HIGH` / `MEDIUM` 指摘は 0 件である。

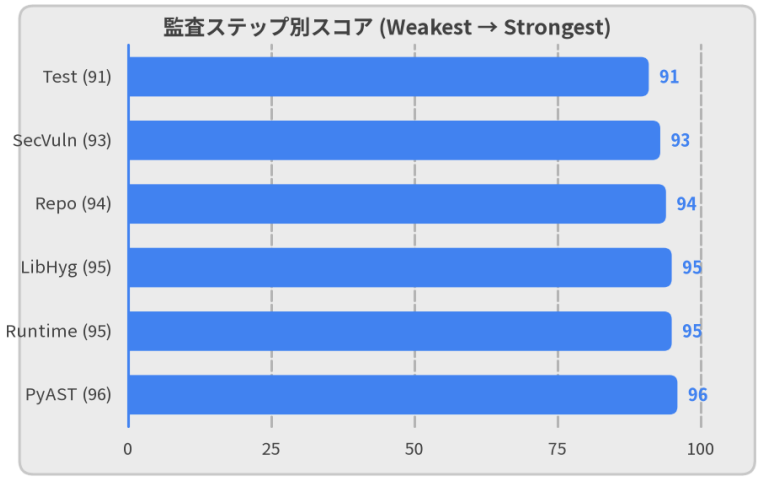




図4-1: 監査ステップ別スコア比較 (昇順: 低スコア順) とルール適合サマリ

4.1 ルール適合マトリクス（全15ルール：`FAIL → WARN → PASS` 順）

判定	ルールID	監査Step	主要な検証エビデンス要約
WARN	common/documentation	act-02	Docstring 100%達成、READMEに関数1件と開発手順の追記余地
WARN	common/security_vulnerability	act-13	uv.lock CVE 0件、コンテナ基盤 pip@25.0.1 に MEDIUM 5件
PASS	common/architecture	act-04	循環依存 0件・境界違反 0件、完全な単方向4層DAG構造
PASS	common/git_workflow	act-02	.gitignore 完備、GitHub Actions CI が main push/PR を保護
PASS	common/library_guidelines	act-08	__all__ 19件明示、内部関数 _ 隠蔽、py.typed、柔軟な下限指定
PASS	common/maintainability	act-04	全6ファイル319 LOC (最大92 LOC)、複雑関数・大域状態 0件
PASS	common/production_readiness	act-11	外部I/Oゼロ、型付き例外階層完備、握り潰し catch ブロック 0件
PASS	common/runtime_verification	act-11	コンテナビルド 5.34s、オフライン pytest 6/6合格、行Cov 96.2%
PASS	common/security	act-02	秘密情報 0件、Pydantic frozen/forbid、Prometheus名サニタイズ
PASS	common/supply_chain	act-02	uv.lock・LICENSE・SECURITY.md・Dependabot・SHA固定CI完備
PASS	python/code_quality	act-04	Docstring 100.0%、シグネチャ不一致 0件、PEP 8 命名準拠
PASS	python/dependencies	act-02	PEP 621 準拠、実行依存 pydantic>=2.7.0 と dev グループ分離
PASS	python/modern_stack	act-04	Import衛生 100.0%、ワイルドカード・未使用依存 0件、StrEnum活用
PASS	python/testing	act-09	3テストファイル・6関数、モジュール網羅 100%、決定論的アサート
PASS	python/typing	act-04	型安全性 100.0%、Any・未スコープ ignore 0件、PEP 561 準拠

図4-2: 全15ルールの適合判定マトリクス (WARN 2件 → PASS 13件)

4.2 重大度順・指摘事項一覧（全5件：`LOW → INFO` 順）

重大度	ルールID	箇所	実証エビデンスと推奨修正
LOW	common/security	models.py:26	allow_inf_nan=False 未設定・isfinite 未検証 → 非有限値拒否を追加
LOW	common/documentation	README.md:37	公開関数1件の記載漏れ・開発手順未記載 → API一覧とコマンドを追記
LOW	security_vulnerability	uv.lock:13	推移的ハッシュ未記録・基盤 pip CVE 5件 → uv lock更新・pip>=26.2.0
INFO	common/supply_chain	ci.yml:18	CI で --locked と uv run ty check 未実行 → ワークフローへ追加
INFO	python/testing	test_rules.py:42	ComparisonOp.LT・単一要素・分散0等の未網羅 → parametrize 追加

図4-3: 重大度順・指摘事項エビデンス一覧 (LOW 3件 → INFO 2件)

指摘事項の補足詳細

1. `[LOW]` `common/security` (`src/metrics_kit/models.py:26`): `ConfigDict(frozen=True, extra="forbid")` に `allow_inf_nan=False` が未指定であり、`stats.py:12` の `_require_non_empty` でも `math.isfinite` を検査していないため、`NaN` / `Inf` 混入時に `sorted(values)` の順序破綻や生の `ValueError` が生じ得る。
2. `[LOW]` `common/documentation` (`README.md:37`): `__init__.py:34` で公開されている `sanitize_prometheus_name` が `README.md` の API Reference に未記載であり、開発・テストコマンド節も不足している。
3. `[LOW]` `common/security_vulnerability` (`uv.lock:13`): `uv.lock` に推移的依存と wheel SHA-256 ハッシュが含まれておらず、コンテナ検証イメージのシステム `pip@25.0.1` に MEDIUM CVE 5件（修正版 `26.2.0`）が検出された。
4. `[INFO]` `common/supply_chain` (`.github/workflows/ci.yml:18`): `pyproject.toml:24` に `ty>=0.0.35` が定義されているが、CI 上で `--locked` フラグおよび `uv run ty check` が未実行である。
5. `[INFO]` `python/testing` (`tests/test_rules.py:42`): `rules.py` の行 `28, 91, 93, 99`（`ComparisonOp.LT`、`z_threshold <= 0`、点数 `< 2`、標準偏差 `0`）および `stats.py`

の行 `66, 109`（単一要素パーセンタイル、空白メトリクス名）の単体テスト追加余地がある。

5. 詳細分析：コード品質・テスト・動的検証

AST静的解析および隔離コンテナ動的検証の結果、全6モジュール・13関数において型安全性・Import衛生・Docstringカバレッジすべて **100.0%**、SIG保守性違反（高複雑度関数・過剰引数・グローバル状態）**0 件**を達成し、オフライン単体テスト `6/6` 合格・実測行カバレッジ **96.2%**（`153/159` statements）を記録した。

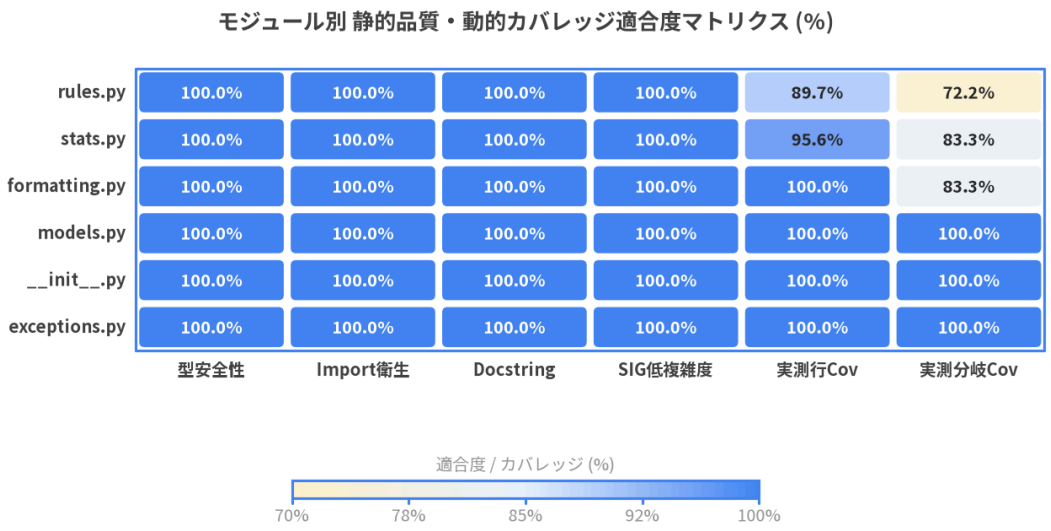


図5-1: モジュール × 品質・カバレッジ適応度マトリクス (HeatmapChart)

5.1 AST 静的コード品質・SIG 保守性・12-Factor 運用適性

- **静的品質カバレッジ (`python_ast_audit`: 96/100)**:
  - `total_files=6`, `total_loc=319`（最大モジュール `stats.py` でも `92 LOC` と高い凝集度を維持）、公開・内部関数 `13` 個・クラス `8` 個。
  - `import_hygiene_coverage_pct=100.0%`（未整列 Import `0`、ワイルドカード Import `0`、未使用・未宣言依存 `0`）。
  - `type_safety_coverage_pct=100.0%`（`Any` 使用 `0`、未スコープ `type: ignore` `0`、PEP 561 `py.typed` マーカー配置済み）。

    - `docstring_coverage_pct=100.0%`（欠落・空 Docstring `0`、引数シグネチャ不一致 `0`）。
- **SIG 保守性・12-Factor 運用準備度**:
    - 循環的複雑度超過関数 `0` 件、過剰パラメータ関数 `0` 件、グローバル可変状態 `0` 件、バグリスクイディオム（ミュータブルデフォルト引数・裸の `except:` 等） `0` 件。
    - 外部 I/O タイムアウト漏れ `0` 件、散在する環境変数参照 `0` 件、非構造化 `print` ログ出力 `0` 件。

5.2 隔離コンテナ動的検証とモジュール別実測カバレッジ

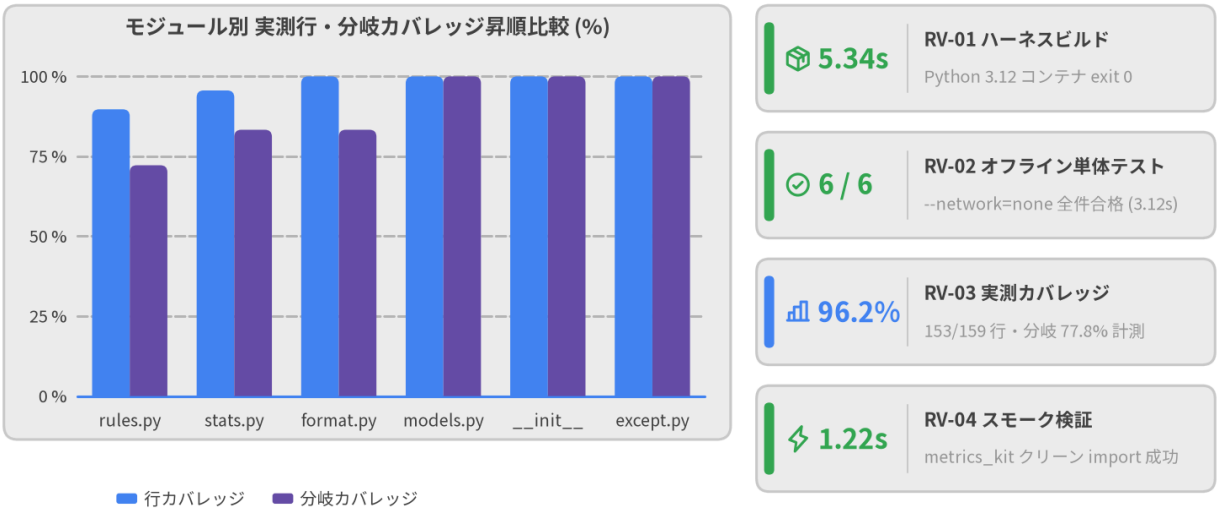
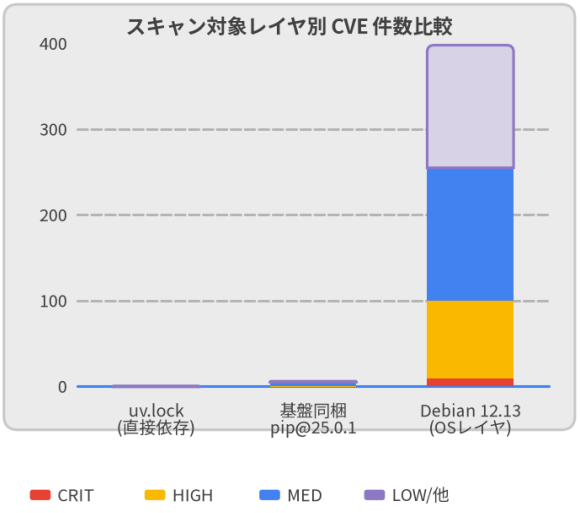


図5-2: モジュール別実測カバレッジ昇順比較 (BarChart) と隔離コンテナ検証ゲート (RV-01〜RV-04)

モジュール（カバレッジ昇順）	ステートメント	実測行カバレッジ	実測分岐カバレッジ	未通過行番号（`missing_lines`）と該当処理
`src/metrics_kit/rules.py`	`35 / 39`	`89.7%`	`72.2%`	`[28, 91, 93, 99]`（`ComparisonOp.LT`、`z_threshold<=0`、点数`<2`、標準偏差`0`）
`src/metrics_kit/stats.py`	`43 / 45`	`95.6%`	`83.3%`	`[66, 109]`（単一要素`compute_percentile`、空白`metric_name`例外分岐）
`src/metrics_kit/formatting.py`	`22 / 22`	`100.0%`	`83.3%`	`[]`（全ステートメント通過、空文字フォールバック分岐1箇所のみ）
`src/metrics_kit/models.py`	`42 / 42`	`100.0%`	`100.0%`（分岐なし）	`[]`（全モデル定義通過）
`src/metrics_kit/__init__.py`	`7 / 7`	`100.0%`	`100.0%`（分岐なし）	`[]`（全公開シンボル再エクスポート通過）
`src/metrics_kit/exceptions.py`	`4 / 4`	`100.0%`	`100.0%`（分岐なし）	`[]`（全例外クラス定義通過）

6. 詳細分析：セキュリティ・脆弱性・API 境界

Trivy（v0.69.3）によるロックファイル・解決済みコンテナイメージスキャンおよび AST SAST 解析の結果、`metrics-kit` 本体と直接依存パッケージ（`pydantic`）の脆弱性は **0 件**、IaC 設定不備 **0 件**、秘密情報漏洩 **0 件**、CWE 危険コードシンク **0 件**、外部 HTTP 攻撃面 **0 件**であることを確認した。



検査対象レイヤ	検出件数	修正版・配布物への影響
uv.lock (直接依存)	0 件	脆弱性ゼロ (配布ホイールは安全)
pip@25.0.1 (基盤同梱)	6 件 (MED 5)	pip>=26.2.0 で解消 (配布物には不含)
debian 12.13 (検証OS層)	398 件	CRIT 9 / HIGH 91 (検証コンテナ限定)
Dockerfile & 全ソース	0 件	IaC設定不備 0件 秘密漏洩 0件

図6-1: レイヤ別脆弱性検出内訳 (uv.lock 直接依存 vs コンテナ同梱 pip vs Debian OS層)

6.1 アプリケーション依存 vs コンテナベースイメージ CVE 詳細と修正バージョン

ライブラリ配布物（`metrics-kit` および `pydantic>=2.7.0`）自体には既知の脆弱性は一切存在しない。解決済みコンテナイメージ上の `python-pkg` 検出 6 件はすべて Python 3.12 ベースイメージ標準添付の `pip@25.0.1` に起因し、OS パッケージ検出 398 件（`debian 12.13`）とともにライブラリ利用者環境へは伝播しないが、CI・検証コンテナの衛生維持のため更新が望ましい。

区分 / 重大度	CVE ID	パッケージ（現行バージョン）	修正バージョン（`fixed_version`）	概要
Python基盤 `MEDIUM`	`CVE-2026-13346` 他計5件	`pip@25.0.1`（コンテナ標準添付）	`26.2.0`	`pip` アーカイブ展開・URL処理等（`CVE-2026-8643`, `6357`, `3219`, `2025-8869` 含む）
Python基盤 `LOW`	`CVE-2026-1703`	`pip@25.0.1`（コンテナ標準添付）	`26.2.0`	`pip` 軽微なパス検証課題
OS層 `CRITICAL`	`CVE-2026-42010` / `33845`	`libgnutls30@3.7.9-2+deb12u5`	`3.7.9-2+deb12u7`	GnuTLS NUL文字認証バイパス / DTLS DoS
OS層 `CRITICAL`	`CVE-2026-31789`（2件）	`libssl3`, `openssl@3.0.18-1~deb12u2`	`3.0.19-1~deb12u2`	32-bit X.509 証明書処理ヒープバッファオーバーフロー（HIGH `CVE-2026-45447` は `3.0.20-1~deb12u2`）
OS層 `CRITICAL`	`CVE-2026-8376` / `13221` / `42496`	`perl-base@5.36.0-7+deb12u3`	`5.36.0-7+deb12u4`	Perl 正規表現ヒープ境界 / `perl-archive-tar` シンボリックリンク走査
OS層 `CRITICAL`	`CVE-2023-45853` / `2025-7458`	`zlib1g@1:1.2.13`, `libsqlite3-0@3.40.1`	未提供（upstream未修正）	`minizip` 整数オーバーフロー / SQLite 整数オーバーフロー
OS層 `HIGH`	`CVE-2026-86145` / `16742`	`libpcre2-8-0@10.42-1`, `libsystemd0`	`10.42-1+deb12u1` / 未提供	PCRE2 境界外書き込み / `systemd-homed` 署名検証課題

6.2 AST 危険コードシンク（CWE SAST）と API 攻撃面マップ



図6-2: AST 危険コードシンク検査 (CWE SAST) と API 境界・サプライチェーン統制マップ